

DOCUMENTACIÓN TÉCNICA

Sistema de Gestión Logística y Optimización de Rutas (Provincia de Córdoba)

1. Arquitectura General y Estructuras de Datos

El sistema está desarrollado bajo el paradigma de Programación Orientada a Objetos (POO) en C++ utilizando el framework Qt para la interfaz gráfica. Los componentes principales estructuran la lógica de la siguiente manera:

Entidad Ciudad (ciudad.h)

Estructura estática básica que representa los nodos (vértices) del mapa provincial.

```
struct Ciudad {  
    int id;  
    char nombre[50];  
    float x;  
    float y;  
};
```

Clase Grafo (grafo.h / grafo.cpp)

Administra la red de conexiones mediante una Matriz de Adyacencia Dinámica (*float** matriz*). Permite verificar la existencia de rutas directas en tiempo constante $O(1)$.

Clase SistemaLogístico (sistemagestionlogistico.h / .cpp)

Funciona como el controlador de la aplicación. Gestiona el arreglo dinámico de ciudades, invoca las funciones del grafo, administra la persistencia binaria y maneja el historial mediante una Pila Dinámica Enlazada (*NodoHistorial**).

2. Algoritmo de Optimización (Dijkstra)

Para determinar el camino mínimo entre un origen y un destino seleccionados, se implementó el algoritmo de Dijkstra dentro de la clase *Grafo*. El proceso opera en base a tres arreglos dinámicos temporales creados en memoria RAM:

- *distancias* (arreglo de *float*): Almacena el costo mínimo acumulado en kilómetros desde el origen a cada ciudad. Se inicializa en infinito (*INF*), excepto el origen que inicia en *0*.
- *visitados* (arreglo de *bool*): Registra qué ciudades ya fueron analizadas para evitar ciclos redundantes.

- **padres** (arreglo de **int**): Guarda el identificador del nodo predecesor óptimo, permitiendo la reconstrucción posterior del recorrido.

Mecánica de Procesamiento Paso a Paso:

1. **Selección del Mínimo:** Se identifica de forma iterativa la ciudad no visitada con la menor distancia acumulada.
2. **Relajación de Aristas:** Se examinan los vecinos directos de dicha ciudad en la Matriz de Adyacencia. Si el costo de viaje hacia un vecino a través de la ciudad actual resulta menor que la distancia previamente registrada, se actualiza el valor en el arreglo **distancias** y se define el nodo actual como el nuevo predecesor en el arreglo **padres**.
3. **Reconstrucción del Camino:** Al alcanzar el destino, el sistema recorre hacia atrás el vector de **padres** desde el nodo final hasta el inicial. Se reserva memoria para el arreglo definitivo del camino resultante (**resultado.camino = new int[contador]**) y se almacena en el orden correcto.
4. **Liberación de Memoria Auxiliar:** Antes de retornar la estructura con el resultado, se destruyen los arreglos temporales mediante operadores **delete[]**.

3. Gestión de Contingencias (Rutas Cortadas)

Para simular bloqueos de infraestructura o contingencias climáticas sin alterar la topología física de la red ni eliminar nodos del sistema, se definió un valor conceptual de infinito:

$$INF = 999999.0f$$

Cuando se reporta un corte de ruta, el método **grafo->cortarRuta(origen, destino)** asigna el valor **INF** a las coordenadas cruzadas correspondientes de la Matriz de Adyacencia. Al ejecutarse una nueva consulta de optimización, el algoritmo de Dijkstra descarta automáticamente ese tramo debido a su costo prohibitivo, obligando al cálculo de una alternativa viable a través de rutas secundarias activas.

4. Visualización Gráfica de Rutas en la Interfaz (Qt)

La representación visual en pantalla se maneja mediante componentes nativos de la vista gráfica de Qt (**QGraphicsScene** y **QGraphicsLineItem**):

- **Estructura Fija:** El mapa provincial consta de un conjunto de rutas preestablecidas declaradas como punteros de líneas en el archivo de cabecera de la ventana principal (**ruta_CP_CC**, **ruta_CC_VM**, etc.).
- **Pintado de Caminos:** Al presionar el botón de cálculo, el método **pintarRutaOptima()** recibe el camino calculado por Dijkstra. La función evalúa de manera secuencial los pares de ciudades consecutivas que integran la solución y actualiza el color de los objetos de línea correspondientes a color verde (**setPen(verde)**), logrando una respuesta visual inmediata sobre el plano gráfico sin necesidad de redibujar elementos de manera dinámica.

5. Persistencia e Historial Binario

La conservación de datos entre sesiones se realiza mediante archivos de flujo binario (*std::fstream*). Los registros se formatean bajo una estructura de tamaño constante en disco para mitigar errores de direccionamiento:

```
struct RegistroHistorial {  
    char origen[50];  
    char destino[50];  
    float distancia;  
};
```

Los datos son volcados y recuperados en bloques limpios de bytes mediante llamadas directas a *archivo.write()* y *archivo.read()*. Durante la inicialización del sistema, el archivo binario se lee de manera secuencial hasta el final del archivo (*EOF*), reconstruyendo la pila dinámica en memoria para su visualización dentro de la aplicación.